upGrad ENTERPRISE

Docker Basics

By
Narasimha Rao T
Microsoft.Net FSD Trainer
Professional Development Trainer
tnrao.trainer@gmail.com

Why Containerization?

1. Problems With Traditional Deployments

Traditional software deployment (installing apps directly on servers/VMs) creates multiple challenges:

a. “Works on my machine” problem

- An app may run perfectly on a developer’s laptop but fail in production.
- Causes: different OS versions, missing libraries, dependency conflicts.

b. Dependency conflicts

- Applications may require:
    - Different versions of the same library.

c. Heavy and slow deployment

- Deployment usually involves:
  - Installing runtime environments
  - Configuring servers manually
  - Long setup times
- Reproducing production environments takes significant effort.

d. Inefficient resource usage

- VMs are heavy:
  - Each VM has its *own OS*
  - Takes lots of CPU, RAM, storage
- Running many VMs on one server reduces performance.

e. Poor scalability

- Scaling apps means:
  - Creating new VMs
  - Installing OS + dependencies
  - Configuring them manually
- Slow and time-consuming.

f. Hard to maintain and update

- Updating apps often requires:
  - Careful, manual steps
  - Risking breaking other apps on the same server.

2. Benefits of Containerization

Containerization solves the above issues by packaging applications with everything they need (code, dependencies, runtime, libraries).

a. Consistency Across Environments

- Containers include all app dependencies.
- Runs the *same* on development, testing, and production.
- Eliminates “works on my machine” issues.

b. Lightweight

- Containers share the host OS kernel.
- No need to boot a full OS.
- Faster startup: milliseconds, not minutes.

c. Fast and portable

- Containers run on:
    - Linux
    - Windows
    - MacOS
    - Cloud platforms (AWS, GCP, Azure)
    - Kubernetes clusters
- You can move an app anywhere without modifying it.

d. Efficient resource usage

- Many containers can run on a single machine.
- Much lighter than running many VMs.

e. Easy scaling

- Need more capacity? Just start more containers.
- Works seamlessly with orchestration tools like Kubernetes.

f. Isolation

- Containers isolate:
  - Processes
  - File systems
  - Network environments
- One container crashing does not affect others.

g. Faster CI/CD

- Build → Test → Deploy pipeline becomes automated and reliable.
- Containers make “immutable deployments” easier.

Docker Basics

Docker is the most widely used containerization platform. It allows developers to build, run, and share containers easily.

1. Containers vs. Virtual Machines

Feature	Containers	Virtual Machines
OS	Shared host OS kernel	Full OS per VM
Size	MBs	GBs
Startup time	Seconds or less	Minutes
Isolation	Process-level	Hardware-level
Performance	Near-native	Slight overhead
Use cases	Microservices, dev environments, scaling apps	Full OS isolation, legacy apps

Analogy

- **VMs** = Full houses with their own plumbing, electricity, etc.

2. Docker Architecture Overview

Docker follows a client–server architecture. It consists of:

1. Docker Engine

The core component that runs and manages containers.

It includes:

- Docker Daemon
- REST API (Docker Engine API)
- Docker CLI

2. Docker Daemon (`dockerd`)

- A background service running on the host OS.
- Responsible for:
  - Building images
  - Running containers
  - Managing networks and volumes
- Listens to Docker API requests from the CLI or other tools.

Key Responsibilities

- Image management
- Container lifecycle management
- Network & volume management

3. Docker CLI (`docker`)

- The command-line interface developers use.
- Sends commands to the Docker Daemon through the REST API.

Common CLI Commands

- `docker run` → start a container
- `docker build` → build an image
- `docker pull` → download an image
- `docker ps` → list running containers
- `docker stop` → stop a container

The CLI doesn't itself run containers — it *talks* to the daemon, which does the actual work.

3. How Docker Components Work Together

1. User runs a command in the CLI:

```
docker run nginx
```

2. CLI sends request via Docker REST API to the daemon.
3. Daemon:
    - Checks if the image exists locally
    - If not, downloads it from Docker Hub
    - Creates a container
    - Starts it
4. Docker Engine isolates the container and allocates resources.

Q & A

Narasimha Rao T
tnrao.trainer@gmail.com